BİTİRME PROJESİ · BÖLÜM 1

minGPT: adım adım

Bir language model'in içini sıfırdan anlamak ve onu bir FPGA üzerinde çalıştırmanın potansiyelini değerlendirmek

Murat Atceken

Ekim 2026

YOL HARİTASI

Bu sunum minGPT'yi sıfırdan kuruyor, FPGA açısından değerlendiriyor ve bir araştırma konusuyla bitiyor

1 **Temel kavramlar**
Language model, token, embedding ve loss: model neyi öğrenir, başarısı nasıl ölçülür.

2 **minGPT nedir?**
GPT-2 mimarisinin 300 satırlık yeniden yazımı: modern LLM'lerden farkı ve kısa kodun sırrı.

3 **minGPT adım adım**
Embedding'in matematiği, LayerNorm, attention, MLP ve output; her adım formül, şekil ve kodla.

4 **Generation ve KV cache**
Model metni nasıl üretir; her adımda yalnızca yeni token'ı işlemek neden yeterlidir.

5 **FPGA potansiyeli**
minGPT'nin hesap yapısı FPGA'ye neden uygun, hangi noktalarda zorlanır.

6 **Araştırma konusu: LittleBit**
Weight'leri low-rank binary factor'larla 0,1 BPW'ye sıkıştırmanın matematiği ve FPGA için anlamı.

Language model, geçmişe bakarak bir sonraki token'ın olasılık dağılımını tahmin eder

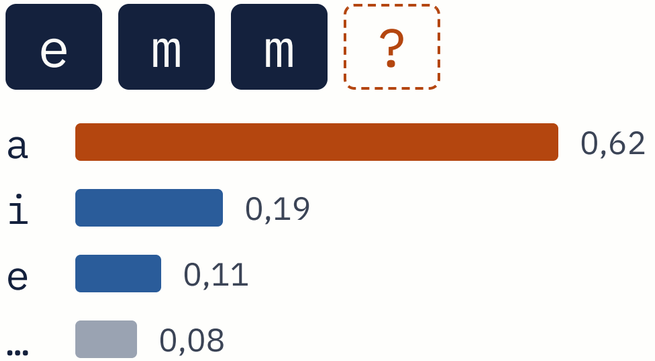
$$p(x_1, \ldots, x_n) = \prod_t p(x_t \mid x_1, \ldots, x_{t-1})$$

Zincir kuralı: bir dizinin olasılığı, her adımdaki "bir sonraki token" olasılıklarının çarpımıdır. Bu bir yaklaşıklık değil, tam bir eşitliktir.

Bu yüzden model tek bir şey öğrenir: şimdiye kadar görülen token'lara bakıp bir sonraki token için bir olasılık dağılımı vermek.

Generation da buradan çıkar: dağılımdan bir token seç, dizinin sonuna ekle, aynı işlemi tekrarla. Buna **autoregressive generation** denir.

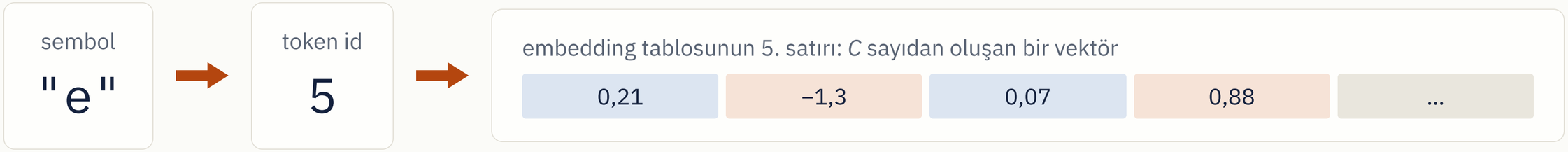
Örnek: isim üreten bir model



Şekil: "emm" görüldükten sonra sonraki harf için dağılım (temsilî değerler, toplamı 1). En yüksek olasılık "a"da: model "emma"yı tamamlamaya yöneliyor.

Token bir sembolün sayı kimliğidir; embedding bu kimliği öğrenilmiş bir vektöre çevirir

Token id yalnızca bir etikettir: “m = 13” harfi “e = 5”in iki katı değildir. Bu yüzden her token, training sırasında öğrenilen *C* sayılık bir vektörle temsil edilir.



Şekil: “e” harfi önce token id 5’e, o da embedding tablosunun 5. satırına dönüşür. Sayılar temsilîdir; gerçek değerler training ile bulunur.

$$\mathbf{e}(\tau) = W_{te}[\tau] = \text{onehot}(\tau)^{\intercal} \cdot W_{te}\ ,\ W_{te}\text{: } V \text{ satır} \times C \text{ sütun}$$

Matematikte bir matris çarpımı, pratikte tek bir satırın okunması. İsim verisinde *V* = 27: 26 harf ve ismin bittiğini gösteren “.” token’ı.

minGPT · model.py

```
wte = nn.Embedding(vocab_size, n_embd)
tok_emb = self.transformer.wte(idx)
# idx: token id’ler → (B, T, C) vektörler
```

Model, doğru token'a verdiği olasılıkla ölçülür (cross-entropy loss) ve bu loss'u küçültecek şekilde eğitilir

$$\mathcal{L} = -\,(1/N)\,\Sigma_t \ln p(\text{doğru token})$$

Her pozisyonda doğru token'a verilen olasılığın eksi logaritması, ortalaması alınır. Birimi nat/token; küçük olan daha iyi.

Doğru token'a verilen olasılık	loss	yorum
1/27 ≈ 0,037 (her harfe eşit)	3,30	hiçbir şey bilmiyor
0,5	0,69	iki aday arasında
0,9	0,11	emin ve haklı
1	0	mükemmel

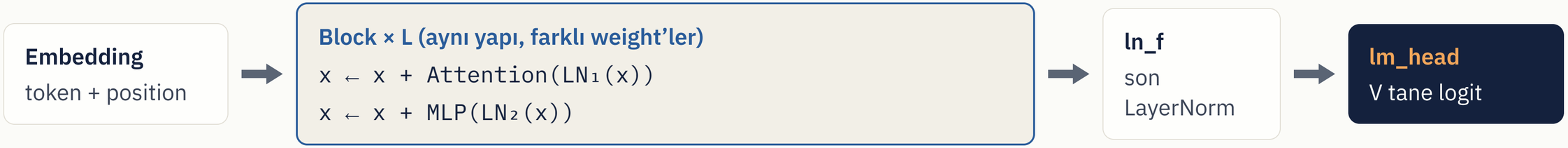
minGPT · trainer.py: training döngüsünün kalbi

```
logits, self.loss = model(x, y) # loss
model.zero_grad(set_to_none=True)
self.loss.backward() # gradient
self.optimizer.step() # AdamW adımı
```

Training ve inference ayrımı

Training bilgisayarda, floating point ile yapılır. Bir FPGA'de yalnızca eğitilmiş modeli çalıştırırız: buna inference denir.

minGPT'nin tamamı üç parça: embedding, L tane aynı block ve bir output katmanı



Şekil: bir token'ın yolu soldan sağa. Her block iki alt adım yapar: attention (token'lar arası bilgi alışverişi) ve MLP (her token'ın kendi içinde hesap). İkisinin sonucu da residual connection ile x'e eklenir.

minGPT · GPT.forward: şemadaki her kutu bir satır

```
tok_emb = self.transformer.wte(idx)
pos_emb = self.transformer.wpe(pos)
x = self.transformer.drop(tok_emb + pos_emb)
for block in self.transformer.h:
    x = block(x)
x = self.transformer.ln_f(x)
logits = self.lm_head(x)
```

Bu sunumdaki model: gpt-nano

$L = 3$ block, $H = 3$ attention head

$C = 48$ (vektör boyutu), $d_h = 16$ (head başına)

$V = 3$ token, $T = 11$ (en uzun context)

Toplam ≈ 85,7 bin parametre.

2 · MİNGPT NEDİR?

minGPT yeni bir mimari değil: GPT-2'nin eğitim amaçlı, sade bir yeniden yazımı

Bileşen	minGPT (GPT-2 mimarisi, 2019)	Modern LLM'ler (ör. Llama 2, nanochat)
Normalization	LayerNorm, pre-LN düzeni	RMSNorm (ortalama çıkarılmaz, daha basit)
Pozisyon bilgisi	learned position embedding	RoPE: q ve k vektörlerini açıyla döndürür
MLP	GELU, genişlik 4*C*	SwiGLU veya ReLU²
Attention	multi-head attention	GQA (head'ler key/value paylaşır), QK-norm
Bias	her Linear'da var	yok
Generation	KV cache yok: her adımda tüm prefix	KV cache: yalnız yeni token işlenir
Ölçek	gpt-nano: 85,7 bin parametre	milyonlarca ile milyarlarca parametre

Asıl mesaj

minGPT, OpenAI'nin GPT-2 weight'lerini from_pretrained() ile doğrudan yükleyebiliyor; bu ancak mimari birebir aynıysa mümkün. Çekirdek mekanizma bütün bu modellerde aynı: decoder-only transformer. Modern LLM'ler bu çekirdeğin ince ayarlı ve çok büyütülmüş hâli.

310 satır neden yetiyor? Ağır işi PyTorch yapıyor, transformer da kendini tekrar ediyor

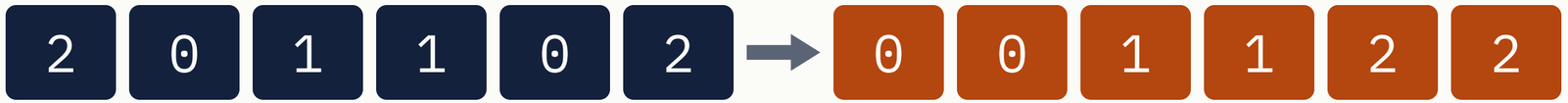
1. Gradient'leri PyTorch otomatik hesaplar (autograd); backpropagation tek satır:
   `self.loss.backward()`
2. Tek bir Block sınıfı yazılır ve L kez kopyalanır:
   `nn.ModuleList([Block(config) for _ in range(config.n_layer)])`
3. Query, key ve value tek bir Linear ile, tek seferde hesaplanır:
   `q, k, v = self.c_attn(x).split(self.n_embd, dim=2)`
4. Bütün head'ler döngü yazmadan, tensor'ü yeniden şekillendirip tek matris çarpımıyla işlenir:
   `k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2)`
5. Causal mask sayesinde bir dizideki T pozisyonun hepsi tek forward'da, paralel eğitilir:
   `att.masked_fill(self.bias[:,:,:T,:T] == 0, float('-inf'))`

Bilerek dışarıda bırakılanlar

- KV cache
- FlashAttention gibi hızlı kernel'ler
- mixed precision
- çoklu GPU ile training

Amaç hız değil, okunabilirlik. model.py yorumlarla birlikte 310 satır.

Çalışan örneğimiz: gpt-nano altı rakamı küçükten büyüğe sıralamayı öğreniyor



Şekil: lacivert kutular girdi (her biri 0, 1 veya 2 olan altı rakam), turuncu kutular modelin üretmesi gereken sıralı çıktı. Model ikisini tek bir token dizisi olarak görür.

minGPT · demo.ipynb, SortDataset: eğitim örneği nasıl kurulur

```
sol = torch.sort(inp)[0] # sıralı çıktı
cat = torch.cat((inp, sol), dim=0) # 12 token
x = cat[:-1].clone() # modele giren 11 token
y = cat[1:].clone() # hedef: bir sonraki token
y[:self.length-1] = -1 # girdi kısmında loss yok
```

$3^6 = 729$ girdi

Olası bütün girdiler sayılabilecek kadar az; modeli hepsinde, örnekleme yapmadan test edebiliyoruz.

%100 doğru

Eğitilmiş gpt-nano, 729 girdinin hepsini doğru sıralıyor.

Adım 1 · Token embedding ile position embedding toplanır: vektör hem “ne” hem “nerede” bilgisini taşır

$W_{te}[\tau_t]$ · token embedding: token ne? + $W_{pe}[t]$ · position embedding: nerede? = $x_t^{(0)}$ · block’lara giren vektör



Şekil: iki öğrenilmiş vektör eleman eleman toplanır (renkler sayıların temsilidir). Böylece aynı token, farklı pozisyonlarda farklı bir vektörle temsil edilir.

$$x_t^{(0)} = W_{te}[\tau_t] + W_{pe}[t] \text{ , } \tau_t\text{: } t \text{ pozisyonundaki token}$$

Position neden gerekli? Attention geçmişteki vektörlerin ağırlıklı toplamıdır ve toplama sırayı bilmez. Position eklenmeseydi “dog bites man” ile “man bites dog” (aynı üç token, farklı anlam) ya da “2 0 1” ile “1 0 2” modele aynı görünürdü.

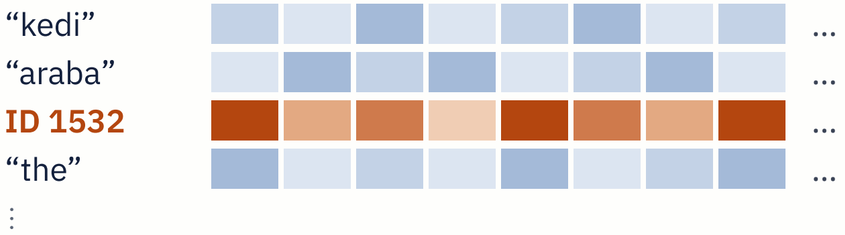
Dropout yalnızca training’de etkindir; inference’ta hiçbir şey yapmaz.

minGPT · GPT.forward

```
pos = torch.arange(0, t, dtype=torch.long)
tok_emb = self.transformer.wte(idx) # ne?
pos_emb = self.transformer.wpe(pos) # nerede?
x = self.transformer.drop(tok_emb + pos_emb)
```

Embedding matrisi: vocabulary'deki her token'ın, training ile öğrenilen bir satırı vardır

E ∈ ℝ^(V × d): *V* satır, *d* sütun



Şekil: her satır bir token'ın vektörü. ID 1532'nin vektörü: $e_{1532} = E[1532]$ = [0,17; −0,42; 0,09; ...; 0,81] (temsilî).

$$\text{token ID } x \longrightarrow \mathbf{e}_x = E[x] \in \mathbb{R}^{\mathbf{d}}$$

Örnek: *V* = 50 000, *d* = 4096 ⇒ *E*'de 205 milyon sayı. gpt-nano'da *V* = 3, *d* = 48.

Bir cümle bir matris olur: "Murat yapay zekayı seviyor" → ID'ler [142, 891, 6251, 211] (temsilî) →

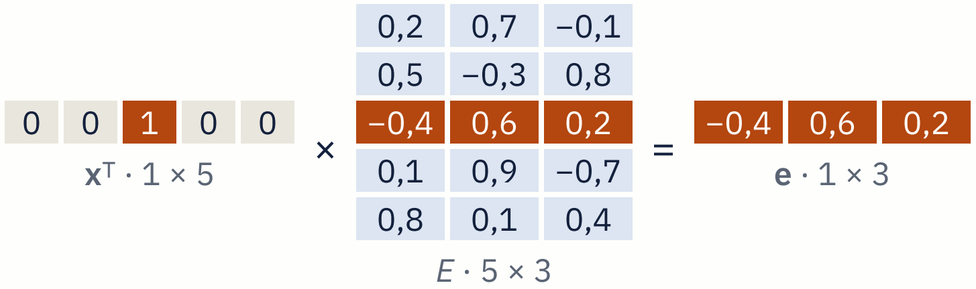
$$X = [E[142]; E[891]; E[6251]; E[211]] \in \mathbb{R}^{(4 \times 4096)}$$

Batch ile: $X \in \mathbb{R}^{(B \times T \times d)}$, örneğin 32 × 2048 × 4096. Transformer'ın gördüğü giriş tam olarak budur.

Tek tek sayıların insanlar için doğrudan bir anlamı yoktur; anlam, vektörlerin birbirine göre konumundadır.

Embedding lookup aslında bir matris çarpımıdır: one-hot vektör, tablonun tek bir satırını seçer

Sayısal örnek: $V = 5$, $d = 3$, 3. token



Şekil: one-hot vektördeki tek 1, E'nin 3. satırını seçer; diğer satırlar 0 ile çarpılıp kaybolur.

$\mathbf{x} \in \mathbb{R}^V$, yalnızca i'nci eleman 1

$$\mathbf{e} = \mathbf{x}^T E = \Sigma_j \, x_j \, E[j, :] = E[i, :]$$

boyutlar: $(1 \times V)(V \times d) = 1 \times d$

Toplamdaki bütün terimler 0 ile çarpılır; geriye yalnızca i'nci satır kalır.

PyTorch: iki yol, aynı sonuç (gpt-nano'da denendi)

```
x1 = self.transformer.wte(idx) # satır okuma
x2 = F.one_hot(idx, V).float() @ wte.weight # matris çarpımı
torch.allclose(x1, x2) # True
```

Maliyet

Çarpım olarak $V \cdot d$ MAC; okuma olarak d sayı, 0 çarpma. Bu yüzden kod embedding[token_id] yazar.

Neden one-hot'ta kalmıyoruz? One-hot vektörlerin hepsi birbirine eşit uzaklıktadır, benzerlik bilgisi taşımaz

One-hot

```
kedi = [1, 0, 0]
köpek = [0, 1, 0]
masa = [0, 0, 1]
```

çift	uzaklık
kedi – köpek	$\sqrt{2} \approx 1{,}41$
kedi – masa	$\sqrt{2} \approx 1{,}41$

Her $i \neq j$ için $\|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{2}$. "Kedi köpeğe masadan daha benzer" bilgisi temsil edilemez.

Embedding (temsilî, $d = 3$)

```
kedi = [ 0,80; 0,70; 0,10]
köpek = [ 0,75; 0,68; 0,15]
masa = [−0,20; 0,10; 0,90]
```

çift	uzaklık	cosine
kedi – köpek	**0,07**	**1,00**
kedi – masa	1,41	0,00

Kedi ile köpek aynı yönde, kedi ile masa dik. Benzerlik geometriye dönüştü.

$\cos\theta = x \cdot y \,/\, (\|x\|\,\|y\|)$ 1 aynı yön, 0 dik, −1 zıt yön. Embedding'in amacı: **anlamsal yakınlık ≈ geometrik yakınlık**.

Embedding öğrenilmiş bir koordinat sistemidir: eksenlerin anlamını biz değil, training belirler

Oyuncak örnek, $d = 3$

token	canlılık	insanlık	mekân
kedi	0,95	0,10	0,05
köpek	0,93	0,08	0,04
doktor	0,90	0,95	0,10
ev	0,05	0,05	0,96

Şekil: eksenlerin anlamı bizim varsayımımız. Cosine: kedi–köpek 1,00, kedi–doktor 0,76, kedi–ev 0,11.

Distributed representation

Gerçek modellerde (d = 2048, 4096, 8192…) bir kavram tek bir eksende durmaz: "canlılık" ≈ 0,2 e_{4} − 0,7 e_{81} + 0,13 e_{946} + … (temsilî). 4096 boyut 4096 kavram demek değildir.

Yönler anlam taşıyabilir

$$\mathbf{e}(\text{king}) - \mathbf{e}(\text{man}) + \mathbf{e}(\text{woman}) \approx \mathbf{e}(\text{queen})$$

Word2Vec'in ünlü örneği (Mikolov vd., 2013): bazı ilişkiler yaklaşık doğrusal yönlerdir. Modern LLM'lerin temsilleri bu kadar basit aritmetikle açıklanamaz.

d neden büyük?

Dilde çok sayıda gizli özellik var: syntax, anlam, zaman kipi, dil, alan, dünya bilgisi ve bunların kombinasyonları.

Embedding nasıl öğrenilir? Gradient yalnızca dizide geçen token'ların satırlarına gider

Başlangıçta rastgele; next-token prediction ile güncellenir: $E \leftarrow E - \eta \cdot \partial \mathcal{L}/\partial E$

$$x_t = W_{te}^{\top}\,\mathbf{e}(\tau_t) + W_{pe}[t] \Rightarrow \partial \mathcal{L}/\partial W_{te} = \Sigma_t\,\mathbf{e}(\tau_t)\,(\partial \mathcal{L}/\partial x_t)^{\top}$$

$$\partial \mathcal{L}/\partial W_{te}[i,:] = \Sigma\,\{\,\partial \mathcal{L}/\partial x_t : \tau_t = i\,\}\,,\ \partial \mathcal{L}/\partial W_{pe}[t,:] = \partial \mathcal{L}/\partial x_t$$

Okunuşu: bir token satırının gradient'i, o token'ın geçtiği bütün pozisyonlardaki gradient'lerin toplamıdır. Dizide geçmeyen token'ın satırına gradient gitmez (sparse gradient). x bir toplam olduğu için aynı $\partial \mathcal{L}/\partial x_t$ hem token satırına hem position satırına aynen gider.

minGPT · model.py: başlangıç ve weight decay

```
elif isinstance(module, nn.Embedding):
    torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
blacklist_weight_modules = (torch.nn.LayerNorm, torch.nn.Embedding)
```

Örnek: girdi 2 0 1 1 0 2

satır	gradient toplanan pozisyonlar
$W_{te}[0]$	t = 1 ve 4
$W_{te}[1]$	t = 2 ve 3
$W_{te}[2]$	t = 0 ve 5

Başlangıç: her eleman N(0, 0,02²). Training sonrası standart sapma: token tablosu 0,049, position tablosu 0,060.

Weight decay: embedding'lere uygulanmaz (blacklist).

Gerçek modelde: gpt-nano'nun position embedding'leri, girdi ve çıktı bölgelerini kendiliğinden ayırmış

t	0	1	2	3	4	5	6	7	8	9	10
0	1,00	0,45	0,53	0,38	0,43	0,12	−0,43	−0,53	−0,29	−0,20	0,06
1	0,45	1,00	0,65	0,67	0,64	0,27	−0,60	−0,55	−0,40	−0,21	−0,07
2	0,53	0,65	1,00	0,80	0,73	0,39	−0,52	−0,61	−0,52	−0,24	−0,07
3	0,38	0,67	0,80	1,00	0,85	0,41	−0,56	−0,72	−0,49	−0,16	0,01
4	0,43	0,64	0,73	0,85	1,00	0,56	−0,56	−0,68	−0,51	−0,31	−0,19
5	0,12	0,27	0,39	0,41	0,56	1,00	−0,09	−0,40	−0,58	−0,59	−0,72
6	−0,43	−0,60	−0,52	−0,56	−0,56	−0,09	1,00	0,62	0,09	−0,19	−0,29
7	−0,53	−0,55	−0,61	−0,72	−0,68	−0,40	0,62	1,00	0,54	0,10	−0,12
8	−0,29	−0,40	−0,52	−0,49	−0,51	−0,58	0,09	0,54	1,00	0,66	0,28
9	−0,20	−0,21	−0,24	−0,16	−0,31	−0,59	−0,19	0,10	0,66	1,00	0,66
10	0,06	−0,07	−0,07	0,01	−0,19	−0,72	−0,29	−0,12	0,28	0,66	1,00

Şekil: 11 position vektörünün ikişerli cosine benzerliği, $\cos(a, b) = a \cdot b \,/\, (\|a\| \, \|b\|)$. Turuncu aynı yön, mavi ters yön. 0–5 girdi, 6–10 çıktı pozisyonları.

Girdi bölgesi tek parça: 1–4 pozisyonları birbirine çok benziyor (0,64–0,85). Sıralamada girdinin sırası önemsiz; model bunu öğrenmiş.

İki bölge zıt yönlü: girdi (1–4) ile çıktı (6–8) pozisyonları arasında −0,40 ile −0,72.

Çıktıda sıra kodlanmış: komşu çıktı pozisyonları benziyor (0,54–0,66).

Uzunluklar: $\|W_{pe}[t]\|$ girdi pozisyonlarında 0,16–0,20, çıktı pozisyonlarında 0,46–0,72.

Token embedding'leri: cos(0, 1) = −0,40, cos(0, 2) = −0,50, cos(1, 2) = −0,07.

Pozisyon bilgisi üç yolla verilir: learned tablo, sinusoidal formül ya da RoPE ile döndürme

Learned · minGPT, GPT-2

$x_t = W_{te}[\tau_t] + W_{pe}[t]$

$T \times C$ parametre, training ile öğrenilir. Tablo T satırlı olduğu için daha uzun context'e genelleşmez.

Sinusoidal · Vaswani 2017

$\mathrm{PE}(t, 2i) = \sin(t \,/\, 10000^{(2i/C)})$

$\mathrm{PE}(t, 2i{+}1) = \cos(t \,/\, 10000^{(2i/C)})$

Parametre yok. Her boyut çifti farklı frekansta dönen bir saat gibi.

RoPE · Llama, nanochat

$q' = R(t\theta_i)\, q\ ,\ \theta_i = 10000^{(-2i/d_h)}$

x'e değil, q ve k'nin her $(2i, 2i{+}1)$ çiftine uygulanır; 2×2 döndürme matrisi R.

RoPE neden işe yarar? Bir satırlık ispat

$$(R(t\theta)\, q)^{\mathsf{T}}\, (R(s\theta)\, k) = q^{\mathsf{T}}\, R(t\theta)^{\mathsf{T}}\, R(s\theta)\, k = q^{\mathsf{T}}\, R((s-t)\theta)\, k$$

Döndürme matrisleri için $R(a)^{\mathsf{T}}R(b) = R(b-a)$. Score yalnızca göreli uzaklığa $(s-t)$ bağlı kalır.

Donanım açısından

Learned position bir tablo okuması ve bir toplama. RoPE ise her token'da her head için cos/sin değerleriyle çarpma ister.

Token embedding sabittir; layer'lar ilerledikçe aynı token'ın vektörü bağlama göre değişir

"I deposited money in the **bank**." → para yatırılan banka

"He sat on the river **bank**." → nehir kıyısı

$$h_i^{(0)} = E[x_i] + p_i \rightarrow H^{(l+1)} = F_l(H^{(l)}) \rightarrow h_i^{(L)} = f(x_i; x_1, \ldots, x_n)$$

attention: $z_i = \Sigma_j \alpha_{i,j} v_j \Rightarrow$ **token kimliği + bağlam bilgisi**

İki cümlede "bank" başlangıçta aynı satırdan ($h^{(0)}$ aynı token vektörü), son layer'da ise farklı vektörlerle temsil edilir.

Token embedding

Transformer'a girmeden önce: *E*'nin satırı. Bir token için her cümlede aynıdır.

Contextual

L layer sonra: $h_i^{(L)}$ bütün cümleye bağlıdır. LLM'lerin gücünün kaynağı budur.

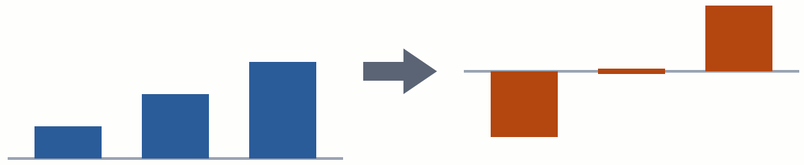
Sentence embedding

RAG'de ayrı bir model bütün metni tek vektöre çevirir; arama cosine ile yapılır. Token embedding ile aynı şey değildir.

Adım 2 · LayerNorm, her token vektörünü ortalaması 0 ve varyansı 1 olacak şekilde yeniden ölçekler

$$\mu = (1/C)\ \Sigma_i\ x_i\ ,\ \sigma^2 = (1/C)\ \Sigma_i\ (x_i - \mu)^2$$
$$\mathrm{LN}(x) = \gamma \odot (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)} + \beta$$

Neden? Layer'lar ilerledikçe vektörlerin büyüklüğü kontrolsüz büyüyüp küçülebilir. LayerNorm bunu her alt adımın girişinde sabitler ve training'i kararlı kılar. γ (scale) ve β (shift) öğrenilen parametrelerdir.



x = [2, 4, 6]
μ = 4, σ ≈ 1,63

LN(x) ≈ [−1,22; 0; 1,22]
μ = 0, σ = 1

Şekil: aynı üç sayı, normalize edilmeden önce ve sonra (γ = 1, β = 0).

minGPT · Block: pre-LN düzeni

```
self.ln_1 = nn.LayerNorm(config.n_embd)
self.ln_2 = nn.LayerNorm(config.n_embd)
x = x + self.attn(self.ln_1(x)) # norm, attention'dan önce
x = x + self.mlpf(self.ln_2(x)) # norm, MLP'den önce
```

FPGA için not

LayerNorm ortalama, karekök ve bölme içerir. Bu, donanımda hesaplaması en pahalı adımlardan biridir.

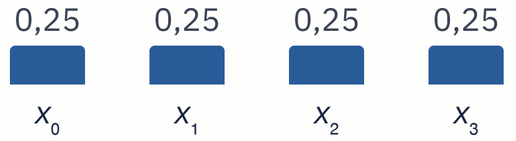
Attention neden gerekli? Her token'ın, önceki token'lardan bilgi toplaması gerekiyor

Embedding'ten sonra her vektör yalnızca kendi token'ını ve pozisyonunu bilir. Ama sonraki token'ı tahmin etmek için geçmişe bakmak gerekir. Diyelim ki 4. pozisyondayız: $x_0 \ldots x_3$'ten tek bir y_3 vektörü nasıl toplanır?

En basit yol: geçmişin eşit ağırlıklı ortalaması

$$y_3 = \tfrac{1}{4}\,(x_0 + x_1 + x_2 + x_3)$$

Her geçmiş token'a aynı önem verilir; model neye bakacağını seçemez.



Attention: içeriğe göre ağırlıklı ortalama

$$y_3 = 0{,}05\,x_0 + 0{,}70\,x_1 + 0{,}05\,x_2 + 0{,}20\,x_3$$

Weight'leri model, token'ların içeriğine bakarak kendisi hesaplar; ilgili token'a daha çok bakar.



Tanım: attention, geçmişteki token vektörlerinin, weight'leri içeriğe göre hesaplanan bir ağırlıklı ortalamasıdır. Weight'ler pozitiftir ve toplamları 1'dir. (Çubuklardaki değerler temsilî.)

Attention'ın fikri: bir sözlükte arama yapmanın "yumuşak" bir versiyonu

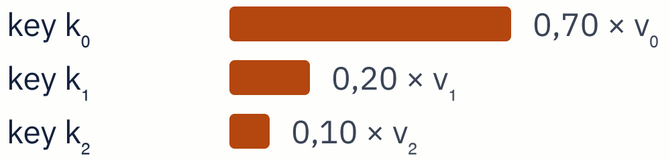
Sert arama: Python dict

```
d = {"kedi": v0, "köpek": v1, "kuş": v2}
y = d["kedi"] # tek bir key eşleşir
```



Yumuşak arama: attention

```
w = softmax([q @ k for k in keys])
y = sum(wi * vi for wi, vi in zip(w, values))
```



Şekil: dict'te query tek bir key ile birebir eşleşir ve yalnızca onun value'su döner. Attention'da query bütün key'lerle karşılaştırılır; benzerlikler weight'e çevrilir ve value'ların karışımı döner (weight'ler temsilî).

Query · q

"Ne arıyorum?" Şu anki token'ın arama vektörü.

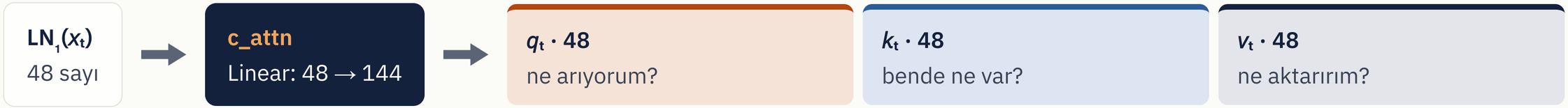
Key · k

"Bende ne var?" Her token'ın kendini tanıtan etiketi.

Value · v

"Seçilirsem ne aktarırım?" Her token'ın içeriği.

Adım 3 · Query, key ve value aynı vektörden, öğrenilen üç linear projection ile üretilir



Şekil: normalize edilmiş vektör tek bir Linear katmandan geçer; çıkan 144 sayı üçe bölünür. Her token'ın kendi query, key ve value vektörü olur.

$$[q_t\,;\,k_t\,;\,v_t] = W\text{attn} \cdot \text{LN}_1(x_t) + b\,,\; W\text{attn}\text{: } 144 \times 48$$

Linear katman nedir? Bir matris-vektör çarpımı ve bir bias: $y_i = \Sigma_j\, W_{ij}\, x_j + b_i$. Her çıkış, girişlerin öğrenilmiş weight'lerle ağırlıklı toplamıdır.

Bu tek satır her token için 144 × 48 = **6 912 çarpma-toplama (MAC)** yapar. Modelin hesabının büyük kısmı bu türden Linear katmanlardır.

minGPT · CausalSelfAttention

```
# __init__: q, k, v için tek Linear
self.c_attn = nn.Linear(n_embd, 3 * n_embd)
# forward: çıkışı üçe böl
q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
```

Adım 4 · Scaled dot-product attention üç işlemdir: score, softmax ve value'ların ağırlıklı toplamı

① $s_{t\,j} = q_t \cdot k_j \,/\, \sqrt{d_h}\,,\; j \le t$

② $p_{t\,j} = \exp(s_{t\,j}) \,/\, \Sigma_i \exp(s_{t_i})$

③ $y_t = \Sigma_j \; p_{t\,j} \; v_j$

Score: query ile her geçmiş key'in dot product'ı; ne kadar benzerlerse o kadar büyük. $\sqrt{d_h}$'ye bölmek, score'ların büyüklüğünü head boyutundan bağımsız tutar.

Softmax: score'ları pozitif ve toplamı 1 olan weight'lere çevirir; büyük score büyük weight demek.

Ağırlıklı toplam: value'ların bu weight'lerle ortalaması; token'ın geçmişten topladığı bilgi.

minGPT · CausalSelfAttention.forward: her satır bir formül

```
att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1))) # ① score
att = att.masked_fill(self.bias[:,:,:T,:T] == 0, float('-inf')) # causal mask (j ≤ t)
att = F.softmax(att, dim=-1) # ② softmax
y = att @ v # ③ ağırlıklı toplam
```

Elle bir örnek: üç geçmiş token, $d_h = 2$ ve x yönünde bir şey arayan bir query

Query q = **[1, 0]**. Üç geçmiş token'ın key ve value vektörleri tabloda. Formüldeki üç işlemi sırayla uyguluyoruz:

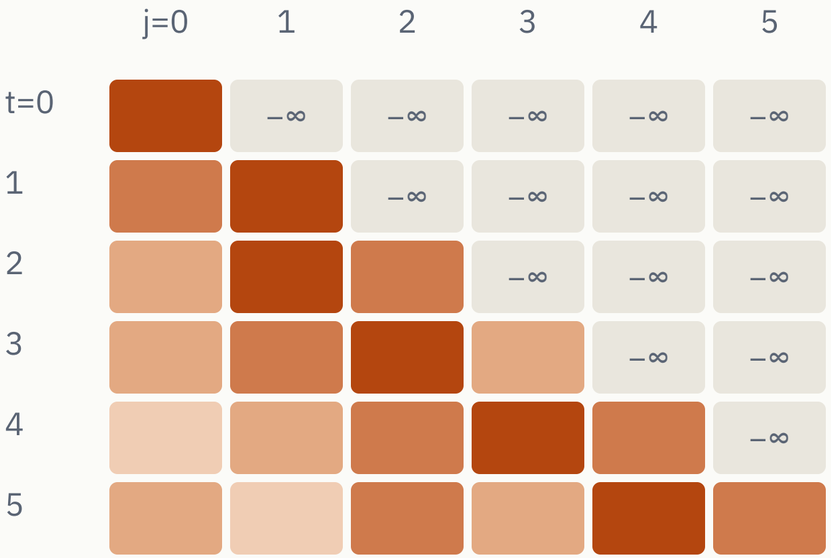
j	key k_j	① $q \cdot k_j$	÷ √2	exp	② weight p_j	value v_j
0	[2, 0]	2	1,41	4,11	**0,58**	[1, 0]
1	[0, 2]	0	0,00	1,00	**0,14**	[0, 1]
2	[1, 1]	1	0,71	2,03	**0,28**	[1, 1]

exp değerlerinin toplamı 7,14. Her weight = exp / 7,14; böylece weight'lerin toplamı tam 1 olur (0,58 + 0,14 + 0,28 = 1).

③ $y = 0{,}58 \cdot [1, 0] + 0{,}14 \cdot [0, 1] + 0{,}28 \cdot [1, 1] =$ **[0,86 ; 0,42]**

Okunuşu: query x yönünü arıyor. En çok x yönlü k_0'a benziyor (weight 0,58), en az y yönlü k_1'e (0,14). Sonuç, ağırlıklı olarak v_0'ın özelliğini taşıyor ama diğerlerinden de pay alıyor.

Causal mask: her token yalnızca kendisine ve geçmişine bakabilir, gelecek tamamen gizlenir



Şekil: satırlar query pozisyonu *t*, sütunlar key pozisyonu *j*. Gri hücreler (*j* > *t*) maskelenir; renkli hücreler softmax weight'leri (tonlar temsilî). Her satırın toplamı 1.

Neden? Training'de model her pozisyonda bir sonraki token'ı tahmin ediyor. Geleceği görebilseydi cevabı doğrudan kopyalardı ve hiçbir şey öğrenmezdi.

Nasıl? *j* > *t* olan score'lar −∞ yapılır. exp(−∞) = 0 olduğu için bu weight'ler yaklaşık değil, tam olarak sıfırdır.

Bonus: Her pozisyon yalnızca kendi geçmişini gördüğü için dizideki bütün pozisyonlar tek forward'da, paralel eğitilebilir.

Sonuç: *t* pozisyonunun çıktısı yalnızca 0…*t* token'larına bağlıdır. KV cache bu gerçeğe dayanır.

```
torch.tril(torch.ones(block_size, block_size))
# alt üçgen: 1 = bakılabilir, 0 = maskeli
```

Gerçek modelde: gpt-nano'nun bir head'i, çıktıyı yazarken girdideki ilgili rakamlara bakıyor

t · token	2	0	1	1	0	2	0	0	1	1	2
t=0 · 2	1,00										
t=1 · 0	0,04	0,96									
t=2 · 1	0,15	0,50	0,35								
t=3 · 1	0,11	0,36	0,26	0,27							
t=4 · 0	0,01	0,37	0,05	0,05	0,53						
t=5 · 2	0,00	0,36	0,02	0,02	0,60	0,00					
t=6 · 0	0,05	0,21	0,16	0,16	0,24	0,08	0,11				
t=7 · 0	0,09	0,12	0,18	0,18	0,13	0,16	0,07	0,06			
t=8 · 1	0,17	0,03	0,15	0,15	0,03	0,37	0,03	0,02	0,04		
t=9 · 1	0,21	0,01	0,10	0,09	0,01	0,51	0,01	0,01	0,02	0,03	
t=10 · 2	0,31	0,00	0,00	0,00	0,00	0,68	0,00	0,00	0,00	0,00	0,01

Şekil: layer 0, head 0'ın softmax weight'leri. Satır *t*: sonraki token tahmin edilirken her geçmiş pozisyona verilen weight. Lacivert girdi, turuncu çıktı.

Girdi **2 0 1 1 0 2**, beklenen çıktı **0 0 1 1 2 2**. Model bu girdiyi doğru sıralıyor.

t = 5 · ilk çıktıyı (0) tahmin ederken

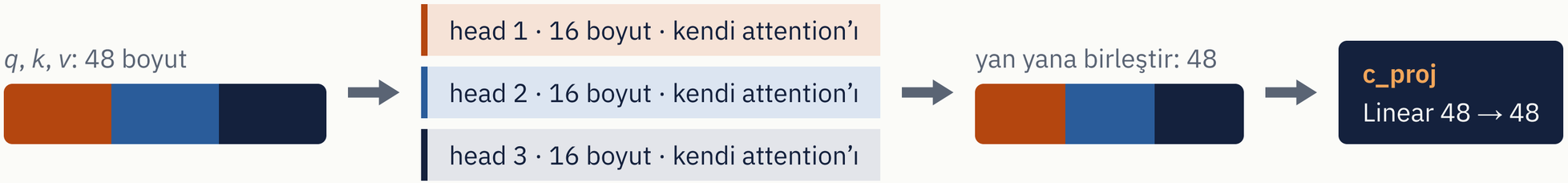
Weight'in %96'sı girdideki iki 0'da: 0,36 + 0,60.

t = 10 · son çıktıyı (2) tahmin ederken

Weight'in %99'u girdideki iki 2'de: 0,31 + 0,68.

Bu davranışı kimse programlamadı; training sırasında kendiliğinden ortaya çıktı. Diğer head'lerin deseni daha az belirgin; iş head'ler ve layer'lar arasında paylaşılıyor.

Adım 5 · Multi-head attention: 48 boyut üç head'e bölünür ve her head kendi ilişkisini arar



Şekil: q, k ve v'nin her biri 16'şar boyutluk üç parçaya bölünür. Her head kendi score'larını ve weight'lerini bağımsız hesaplar; çıktılar yan yana konup c_proj adlı bir Linear ile karıştırılır.

$$y = W\text{proj} \cdot [\ \text{head}_1\ ;\ \text{head}_2\ ;\ \text{head}_3\ ] + b$$

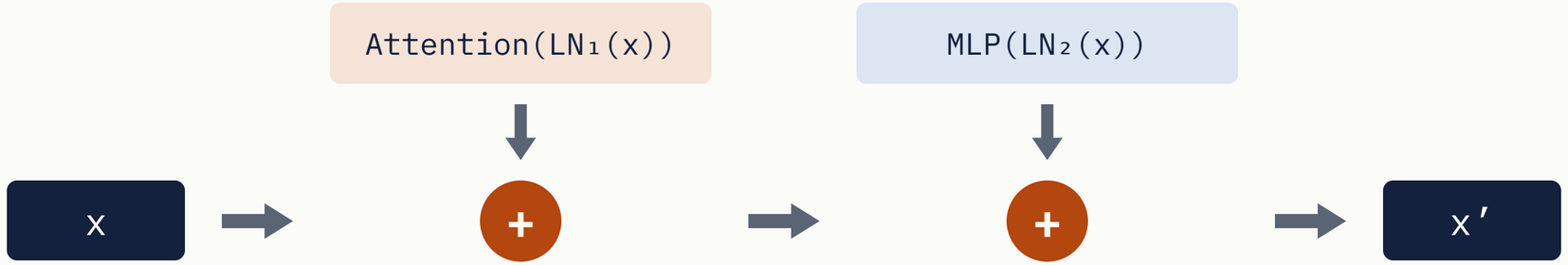
Neden birden çok head? Tek bir head, her pozisyonda yalnızca tek bir ağırlıklı ortalama alabilir. Birden çok head aynı anda farklı ilişkilere bakabilir: biri girdideki küçük rakamlara, bir diğeri bir önceki token'a.

Maliyet: toplam MAC sayısı tek head'li attention ile aynı; vektör yalnızca parçalara bölünür.

minGPT · CausalSelfAttention.forward

```
k = k.view(B, T, nh, C // nh).transpose(1, 2)
# → (B, nh, T, hs): head'ler batch gibi
y = y.transpose(1, 2).contiguous().view(B, T, C)
y = self.resid_dropout(self.c_proj(y))
```

Adım 6 · Residual connection: alt adımlar x'i silmez, üstüne bir düzeltme ekler



Şekil: residual stream soldan sağa akar. Attention ve MLP, stream'in normalize edilmiş bir kopyasını okur ve sonuçlarını stream'e ekler (+). Bir block'ta iki ekleme vardır; L block'ta 2L.

$$x \leftarrow x + f(\mathrm{LN}(x)) \Rightarrow \partial x'/\partial x = I + \partial f/\partial x$$

Training için: türevde birim matris I kalır; gradient derin ağda katman katman küçülüp sönmeden geri akar.

Yorum için: residual stream bütün layer'ların ortak çalışma alanıdır; her alt adım ondan okur ve ona küçük bir düzeltme yazar.

minGPT · Block.forward: block'un tamamı

```
def forward(self, x):
    x = x + self.attn(self.ln_1(x))
    x = x + self.mlpf(self.ln_2(x))
    return x
```

Adım 7 · MLP her token’ı kendi içinde işler: önce dört kat genişletir, GELU’dan geçirir, sonra daraltır

LN$_2$(x) · 48 → c_fc · Linear 48 → 192 → GELU · 192 → c_proj · Linear 192 → 48 → residual’a ekle

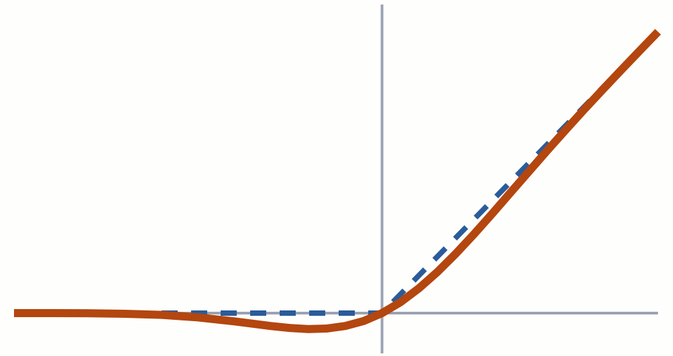
$$\mathrm{MLP}(a) = W_2 \cdot \mathrm{GELU}(W_1\, a + b_1) + b_2$$

$$\mathrm{GELU}(x) = \tfrac{1}{2}\, x\, (1 + \tanh(\sqrt{(2/\pi)}\, (x + 0{,}044715\, x^3)))$$

Neden non-linearity? GELU olmasaydı iki Linear arka arkaya tek bir Linear’a çökerdi. Modelin ifade gücü bu fonksiyondan gelir. Attention bilgiyi token’lar arasında taşır; MLP o bilgiyi her token’da işler.

minGPT · Block.__init__

```
c_fc = nn.Linear(n_embd, 4 * n_embd) # 48 → 192
c_proj = nn.Linear(4 * n_embd, n_embd) # 192 → 48
self.mlpf = lambda x: m.dropout(m.c_proj(m.act(m.c_fc(x))))
```



Şekil: GELU (düz) ve **ReLU** (kesikli), $x \in [-4, 3]$. GELU’nun köşesi yumuşak; negatif tarafta küçük bir çukur var.

Adım 8 · Son LayerNorm ve lm_head her token için bir logit üretir; generation bunlardan birini seçer

token	logit *z*	softmax → olasılık *p*
0	2,2	**0,82**
1	0,4	**0,14**
2	−0,8	**0,04**

Şekil: sıralama görevinde vocabulary 3 token. lm_head her biri için bir logit verir; softmax bunları toplamı 1 olan olasılıklara çevirir (değerler temsilî).

Greedy decoding

En olası token seçilir: 0. Softmax sırayı değiştirmez; argmax(*p*) = argmax(*z*).

Sampling

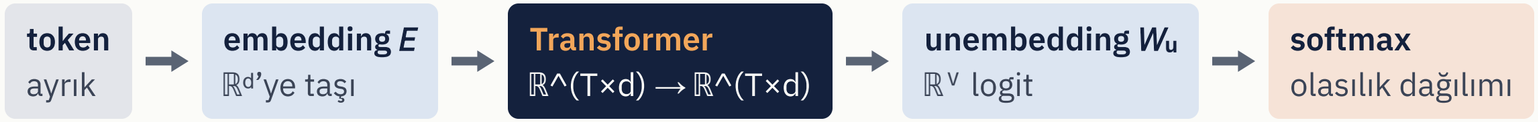
Dağılımdan rastgele çekilir: %82 ihtimalle 0, %14 ihtimalle 1.

minGPT · GPT.generate: her yeni token için

```
logits, _ = self(idx_cond) # tüm prefix!
logits = logits[:, -1, :] / temperature
probs = F.softmax(logits, dim=-1)
if do_sample:
    idx_next = torch.multinomial(probs, num_samples=1)
else:
    _, idx_next = torch.topk(probs, k=1, dim=-1)
idx = torch.cat((idx, idx_next), dim=1)
```

İlk satıra dikkat: minGPT her yeni token için bütün diziyi baştan modelden geçirir. Basit ama israflı; iki slayt sonra bunun neden gereksiz olduğunu göstereceğim.

Çıkış tarafı: unembedding; her logit, son hidden state ile bir token vektörünün dot product'ıdır



$$z = W_u\, h_t\,,\; W_u \in \mathbb{R}^{V \times d} \Rightarrow z_i = u_i^{\top} h_t$$

$$P(x_{t+1} = j) = e^{z_j} \,/\, \Sigma_k\, e^{z_k}$$

Geometrik yorum: model, "şu anki hidden state hangi token'ın yönüyle en uyumlu?" sorusunu çözer. "Paris is the capital of" sonrası h, u(France) ile aynı yöne bakıyorsa z(France) büyük, P(France) yüksek olur.

Weight tying

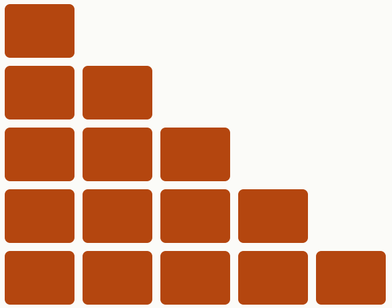
Bazı modellerde $W_u = E$: aynı tablo hem girişte hem çıkışta kullanılır (GPT-2'nin orijinali, llama2.c). minGPT'de lm_head ayrı, bias'sız bir matristir.

minGPT · model.py

```
self.lm_head = nn.Linear(n_embd,
    vocab_size, bias=False)
logits = self.lm_head(x)
```

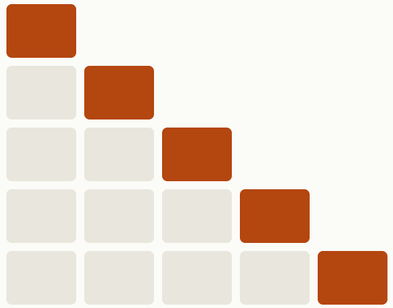
KV cache: geçmiş token'ların key ve value'ları hiç değişmediği için saklanıp yeniden kullanılır

KV cache olmadan



15 token hesabı

KV cache ile



5 token hesabı

Şekil: satırlar generation adımları (1…5), sütunlar token'lar. Turuncu: o adımda baştan hesaplanan token; gri: cache'ten okunan key ve value. Cache'siz iş adım sayısının karesiyle, cache'li iş doğrusal büyür.

Neden doğru? Causal mask yüzünden *j* pozisyonundaki her değer, her layer'da, yalnızca 0…*j* token'larına bağlıdır. Yeni bir token eskileri değiştiremez; bu, layer'lar üzerinden tümevarımla ispatlanır.

Sonuç: her adımda tek token işlenir. Her Linear bir matris-vektör çarpımına, attention da *t*+1 uzunluğunda iki dot product dizisine iner.

Fikir (pseudo-code), layer *l*, adım *t*

```
K[l][t], V[l][t] = k_new, v_new # sakla
s = q_new @ K[l][:t+1].T # score
y = softmax(s) @ V[l][:t+1] # topla
```

5 · FPGA POTANSİYELİ

FPGA, işlevi üretimden sonra tasarlanan bir çiptir ve hesap için dört tür yapı taşı sunar

53 200

LUT

6 girişli küçük doğruluk tablosu. Her mantık fonksiyonu bunlarla kurulur.

106 400

FF (flip-flop)

Tek bitlik register. Ara sonuçları ve pipeline aşamalarını tutar.

140

BRAM (36 Kb)

Çip içi bellek blokları, toplam ~630 KB. Weight’ler burada durabilir.

220

DSP48

Hazır 25×18 bit çarpıcı ve toplayıcı: her clock cycle’da bir MAC.

Sayılar, bu projede kullandığımız PYNQ-Z2 kartındaki Zynq XC7Z020 çipine ait.

PYNQ-Z2: işlemci ve FPGA aynı çipte

Zynq XC7Z020’de çift çekirdekli bir ARM Cortex-A9 işlemci ile programlanabilir mantık yan yana. İşlemci modeli yükler ve sonucu okur; hesap programlanabilir mantıkta yapılır.

GPU’dan farkı

GPU, sabit çekirdeklerde komut yürütür. FPGA’de ise devrenin kendisi tasarlanır: veri yolu, bellek düzeni ve paralellik modele göre biçimlendirilir.

5 · FPGA POTANSİYELİ

minGPT'nin iş yükü: bir token'ın hesabının %96–98'i matris-vektör çarpımı ve model çip içine sığıyor

gpt-nano, KV cache ile, token başına	matris	MAC
q, k, v (c_attn)	144 × 48	6 912
attention çıkışı (c_proj)	48 × 48	2 304
MLP genişletme (c_fc)	192 × 48	9 216
MLP daraltma (c_proj)	48 × 192	9 216
3 block + lm_head	—	**83 088**
attention, layer başına	$2 \cdot C \cdot (t+1)$	96 … 1 056

Linear katmanlar token başına sabit 83 088 MAC yapar; attention pozisyonla büyür ama 11 token'lık context'te toplamın %2–4'ü.

Bellek: 85,7 bin parametre nereye sığar?

BRAM kapasitesi · ~630 KB



weight'ler, 32 bit floating point · ~343 KB



weight'ler, 8 bit · ~84 KB



Şekil: weight'ler 32 bit hâliyle bile çipin BRAM'ine sığıyor; 8 bit ile kapasitenin yalnızca %13'ünü kullanıyor.

Tasarım için iki sonuç

Donanımın kalbi hızlı bir matris-vektör çarpım birimi olmalı. Weight'ler çipin içinde durabildiği için dış belleğe gitmek gerekmez.

Güçlü yanlar: minGPT'nin hesabı küçük, düzenli ve öngörülebilir; bu tam FPGA'nin sevdiği türden

1 · Tek block, L kez

Bütün layer'lar aynı hesabı yapar. Tek bir donanım veri yolu, sırayla bütün layer'ları işleyebilir.

2 · Model çipe sığar

Weight'ler BRAM'de durur; dış DRAM'e gidilmez. Gecikme hem düşük hem sabit olur.

3 · Hesap tek tip

İşin %96–98'i matris-vektör çarpımı. DSP'lerden kurulu tek bir paralel MAC dizisi bunun hepsini yapabilir.

4 · Öngörülebilir akış

Veriye bağlı dallanma yok: her token aynı işlem sırasından geçer. Zamanlama baştan planlanabilir.

5 · Ucuz parçalar

Embedding bir tablo okuması; causal mask donanımda bedava (yalnızca $j \le t$ okunur); GELU tek girişli bir fonksiyon.

6 · Batch = 1'de iyi

Tek kullanıcı, tek token: GPU'ların verimsiz kaldığı bu durumda FPGA'nin özel veri yolu avantajlıdır.

> **Özetle**
>
> Küçük bir FPGA, minGPT ölçeğindeki bir modeli dış belleğe ihtiyaç duymadan, düşük ve sabit gecikmeyle çalıştırmaya uygun.

Zayıf yanlar: non-linear işlemler, büyüyen attention ve floating point için yazılmış bir model

1 · LayerNorm ve softmax

Karekök, üstel fonksiyon ve bölme donanımda pahalı ve yavaş. Bunlar için özel birimler gerekir.

2 · Floating point

Model 32 bit float ile eğitilir. FPGA'de float pahalı; bit genişliğini düşürürken doğruluğu korumak gerekir.

3 · Attention büyür

Her yeni token bütün önceki key'lerle çarpılır. Maliyet pozisyonla doğrusal artar; uzun context'te darboğaz olur.

4 · KV cache belleği

Saklanan key ve value'lar $L \cdot T \cdot C$ kadar büyür. GQA kullanan modellere göre daha çok bellek ister.

5 · Kod GPU için yazılmış

minGPT bütün diziyi aynı anda işler ve KV cache kullanmaz. FPGA için token başına bir hesaba yeniden yazılmalı.

6 · Büyümeye sınır

Büyük modeller çipe sığmaz; o zaman sınır DRAM bant genişliği olur. Learned position da context'i sabitler.

Özetle

Zorlukların hepsi önceden belli ve çözülebilir türden: hangi işlemin pahalı olduğunu, nerede yeniden yazmak gerektiğini biliyoruz. Bu, minGPT'yi FPGA için iyi bir başlangıç noktası yapıyor.

6 · ARAŞTIRMA KONUSU

LittleBit: weight başına 0,1 bit

Bir matrisin her sayısını ayrı ayrı saklamak yerine, onu birkaç ±1 deseni ve birkaç scale ile tarif etmek.

B. Lee, D. Kim, Y. You, Y. Kim · Samsung Research · “LittleBit: Ultra Low-Bit Quantization via Latent Factorization” · NeurIPS 2025 · arXiv:2506.13771

Problem: LLM’in belleği weight’lerden gelir; 1 bitin altına inmek ancak weight’leri tek tek saklamamakla mümkün

$$\text{BPW} = b = (\text{saklanan toplam bit}) \,/\, (d_{out} \cdot d_{in})$$

Bits per weight: matrisi temsil etmek için harcanan bitin, matristeki weight sayısına oranı.

Temsil	BPW	nasıl?
FP16	16	her weight bir sayı
INT8 quantization	8	her weight 8 bit
binary (±1)	1	her weight 1 bit
LittleBit	**0,1 – 1**	matris bir bütün olarak

Ana fikir: her weight için ayrı bir değer saklarsak en az 1 bit gerekir. $b < 1$ için matrisin tamamı, eleman sayısından çok daha az parametreyle temsil edilmelidir.

Llama2-13B’nin belleği (makale, Tablo 3)

FP16 · 26,06 GB



LittleBit 0,3 BPW · 1,15 GB (22,7×)



LittleBit 0,1 BPW · **0,84 GB (31,0×)**



Şekil: sıkıştırma oranı 16/0,1 = 160× değil 31×; çünkü embedding ve lm_head FP16 bırakılıyor.

> **Bizim için:** çipteki ~630 KB BRAM, 0,1 BPW’de kabaca 50 milyon weight demek.

LittleBit’in hilesi: matrisi sayı sayı saklamak yerine birkaç ± desen ve birkaç scale ile tarif etmek

1

Desenlere ayır

Büyük matris, birkaç basit desenin toplamı olarak yazılır. Her desen (rank-1 component) bir sütun ile bir satırın çarpımıdır.



2

Yalnız işareti tut

Desenlerdeki her sayının yalnızca + mı – mi olduğu saklanır: sayı başına 1 bit.



3

Scale’lerle telafi et

Kaybolan büyüklükleri, her satır, her sütun ve her desen için birer scale geri verir: h, g, ℓ.

satır h · sütun g · desen ℓ

Sonuç: 4096 × 4096’lık bir katman 33,6 MB yerine 209 KB tutar (0,1 bit/weight); hesapta çarpma yerine toplama ve çıkarma yapılır.

İki yardımcı fikir: akıllı başlangıç (Dual-SVID) ve ikinci düzeltme yolu (residual).

Hamle 1 · Bir matris, birkaç basit desenin toplamı olarak yazılabilir

1					2	−1	0,5	1
2					4	−2	1	2
−1	×	2	−1	0,5	1	=	−2	1	−0,5	−1
3							6	−3	1,5	3

u (sütun) · v^T (satır, desen) · $W = u\,v^T$ · 16 sayı

16 sayı, ama hepsi 8 sayıdan üretiliyor: her satır aynı desenin (v^T) bir katı. 2. satır = 2 × desen, 3. satır = −1 × desen.

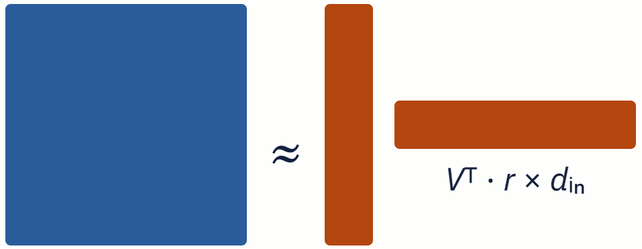
Tek desen (rank 1): $W_{ij} = u_i \cdot v_j$

r desen: $W \approx u_1 v_1^T + u_2 v_2^T + \ldots + u_r v_r^T = U\,V^T$

Gerçek weight matrisleri tek bir desen değildir; ama birkaç desenin toplamına çok yakındır.

Kazanç: $m \cdot n$ sayı yerine $r(m + n)$ sayı. 4096 × 4096'da r = 86: 16,8 M yerine 0,7 M sayı (%4).

Hamle 1 · Matematik: en iyi desenleri SVD bulur; bu, low-rank factorization'dır



Şekil: kare matris (mavi), ince iki matrisin çarpımıyla temsil edilir. İnce kenarın genişliği rank *r*.

Örnek: 4096 × 4096, *r* = 546: 16,8 M sayı yerine 546 × 8192 = 4,47 M sayı (%27).

$$W \approx U\,V^\top,\ U \in \mathbb{R}^{(d_{out} \times r)},\ V \in \mathbb{R}^{(d_{in} \times r)},\ r \ll \min(d_{out}, d_{in})$$

$$\text{SVD: } W = \sum_i \sigma_i\, u_i\, v_i^\top,\ \sigma_1 \geq \sigma_2 \geq \ldots \geq 0,\ W_r = \sum_{i=1}^{r} \sigma_i\, u_i\, v_i^\top$$

$$\text{Eckart–Young: } \min \|W - A\| = \|W - W_r\| = \sqrt{\left(\sum_{i=r+1} \sigma_i^2\right)},\ \operatorname{rank}(A) \leq r$$

Okunuşu: rank-*r* matrisler arasında *W*'ye Frobenius normunda en yakın olanı, SVD'nin ilk *r* terimidir; hata, atılan tekil değerlerin kareleri toplamının köküdür. Tekil değerler hızlı küçülüyorsa az bir *r* ile hata küçük kalır.

Parametre sayısı: $d_{out} \cdot d_{in}$ yerine $r\,(d_{out} + d_{in})$. Yazarlar, LLM weight'lerindeki low-rank yapıya dayanıyor.

Hamle 2 · Desendeki her sayı, bir işaret (1 bit) ile bir scale'in çarpımı olarak yazılır



Şekil: bir önceki slayttaki desenin sayıları büyüklük ve işarete ayrılır. Büyüklükler satır scale'i h = [1, 2, 1, 3] ve sütun scale'i g = [2, 1, 0,5, 1] olur.

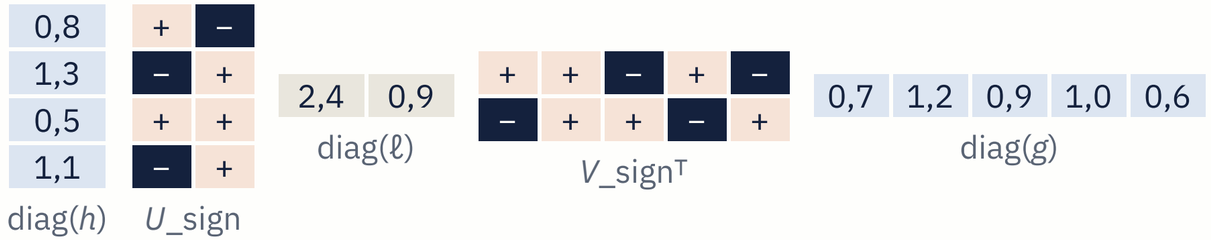
$$W_{ij} = h_i \cdot g_j \cdot \mathrm{sign}(u_i) \cdot \mathrm{sign}(v_j)$$

Örnek: $W_{32} = 1 \cdot 1 \cdot (-1) \cdot (-1) = 1$. Tek bir desen için bu yazım **kayıpsızdır**.

Mikser benzetmesi

± desen sesin hangi yöne gideceğini, scale'ler, yani mikserdeki ses düğmeleri, ne kadar açılacağını söyler. Her satırın (h), her sütunun (g) ve her desenin (ℓ) birer scale'i vardır.

Hamle 2 · Matematik: factor'lar ±1'e iner; büyüklükler satır, sütun ve latent scale'lere (h, g, ℓ) taşınır



Şekil: 4 × 5'lik bir matrisin rank-2 LittleBit temsili (değerler temsilî). Yalnızca ± işaretleri ve üç küçük scale vektörü saklanır.

$$U_\text{sign} = \text{sign}(U),\ V_\text{sign} = \text{sign}(V) \in \{-1, +1\}$$

$$\hat{W} = \text{diag}(h) \cdot U_\text{sign} \cdot \text{diag}(\ell) \cdot V_\text{sign}^{\intercal} \cdot \text{diag}(g)$$

$$\hat{W}_{ij} = h_i\, g_j \sum_k \ell_k \cdot \text{sign}(U_{ik}) \cdot \text{sign}(V_{jk})$$

Okunuşu: her eleman, r tane $\pm\ell_k$ teriminin toplamı; satırına göre h_i, sütununa göre g_j ile ölçeklenir. ℓ, SVD'deki tekil değerlerin rolünü üstlenir.

Saklanan: işaretler $r(d_{out} + d_{in})$ bit, scale'ler $16(d_{out} + d_{in} + r)$ bit (FP16).

Kayıp, birden çok desen aynı satır scale'ini paylaşmak zorunda kalınca doğar

r desen varsa her satırın r tane büyüklüğü vardır: $|U_{i_1}|$, …, $|U_{ir}|$. Ama LittleBit her satır için tek scale (h_i) ve her desen için tek scale (ℓ_k) saklar:

$$|U_{ik}| \approx h_i \cdot \ell_k \text{ (büyüklük tablosunun rank-1 yaklaşımı)}$$

Tablonun gerçek yapısı "satır × desen" çarpımına uymuyorsa hata doğar. Tek desende (r = 1) tablo tek sütundur ve $h = |u|$ alınır: kayıp yoktur.

> **İki çare**
>
> Training scale'leri ve işaretleri birlikte ayarlar. Residual yol, ikinci bir scale takımı ekler.

Örnek: 4 × 4 matris, r = 2 desen

satır	gerçek d_1	gerçek d_2	$h \cdot \ell$ d_1	$h \cdot \ell$ d_2
1	0,76	0,87	0,96	**0,41**
2	1,65	0,77	1,67	0,72
3	0,84	0,80	1,00	**0,43**
4	2,38	0,54	2,20	**0,95**

Şekil: 1. desenin büyüklükleri iyi yaklaşılıyor; 2. desenin profili farklı olduğu için aynı satır scale'leriyle yakalanamıyor (0,87 yerine 0,41).

Dual-SVID: işaretler SVD'den, scale'ler büyüklüklerin rank-1 yaklaşımından gelir

Akıllı başlangıç: SVD'nin bulduğu desenlerden başla, işaretlerini ayır, büyüklük tablosunu "satır scale'i × desen scale'i" olarak yaklaş.

1. Kesik SVD ile başla; tekil değerler iki factor'a paylaştırılır:
   $W \approx U' V'^{\top}$ (rank r)
2. Yönü işaretler taşır:
   $U_\text{sign},0 = \text{sign}(U')$, $V_\text{sign},0 = \text{sign}(V')$
3. Büyüklük matrisleri rank-1 ile yaklaşılır:
   $|U'| \approx h_0 \, \ell_u^{\top}$, $|V'| \approx g_0 \, \ell_v^{\top}$
4. Latent scale'ler birleşir; QAT bu noktadan başlar:
   $\ell_0 = \ell_u \odot \ell_v$, $U \leftarrow U'$, $V \leftarrow V'$

Neden doğru yapıyı verir? Türetme

Anahtar özdeşlik (eleman bazında kontrol edilir):

$$A \odot (h\,\ell^{\top}) = \text{diag}(h) \cdot A \cdot \text{diag}(\ell)$$

$$U' = \text{sign}(U') \odot |U'| \approx \text{diag}(h_0)\; U_\text{sign}\; \text{diag}(\ell_u)$$
$$V' \approx \text{diag}(g_0)\; V_\text{sign}\; \text{diag}(\ell_v)$$
$$\Rightarrow U'V'^{\top} \approx \text{diag}(h_0)\; U_\text{sign}\; \text{diag}(\ell_0)\; V_\text{sign}^{\top}\; \text{diag}(g_0) = \hat{W}_0$$

$\text{diag}(\ell_u)\,\text{diag}(\ell_v) = \text{diag}(\ell_u \odot \ell_v)$ olduğu için ℓ_0 tek bir vektöre iner. Training, LittleBit yapısının içinde SVD çözümüne yakın bir noktadan başlar.

Elle örnek: 4 × 4’lük bir matriste LittleBit ne kadar yaklaşıyor ve residual neden işe yarıyor?

orijinal W			
2,4	−0,3	1,2	0,3
3,6	−2,7	0,3	2,7
−1,6	1,7	0,2	−1,7
6,2	−2,6	1,8	2,6

$\hat{W}$: tek yol, $r = 2$			
2,62	−1,06	0,90	1,06
3,12	−2,69	1,07	2,69
−1,88	1,62	−0,65	−1,62
6,00	−2,42	2,07	2,42

fark $W - \hat{W}$			
−0,22	0,76	0,30	−0,76
0,48	−0,01	−0,77	0,01
0,28	0,08	0,85	−0,08
0,20	−0,18	−0,27	0,18

$\hat{\mathbf{W}}$, Dual-SVID başlangıcıdır: işaretler ve scale’ler SVD’den. Farkın büyük olduğu hücreler, bir önceki slayttaki 2. desenin yakalanamayan kısmıdır.

Temsil (aynı 4 × 4 matris)	göreli hata $\|W - \hat{W}\| / \|W\|$
SVD, rank 2 (gerçek sayılarla, sıkıştırmasız)	%0,4
LittleBit, tek yol, $r = 2$ · Dual-SVID başlangıcı	%17,7
aynı yapı, scale’ler ve işaretler eğitilince*	%14,5
ana yol + residual yol, her biri $r = 1$ · başlangıç	**%0,4**

* Bizim oyuncak hesabımız: makaledeki loss yerine doğrudan $\|W - \hat{W}\|^2$ küçültüldü (Adam, SmoothSign). Her yol tek desen taşıyınca işaret + scale kayıpsızdır. 4 × 4’te bit kazancı yok; kazanç büyük matrislerde.

Hamle 3 · Ŵ hiç kurulmaz: y = Ŵx yalnızca toplama, çıkarma ve birkaç scale çarpımıyla bulunur

$x \cdot d_{in}$ → $\odot g$ → $V_sign^T \cdot \pm 1$ ile topla → $\odot \ell$ → $U_sign \cdot \pm 1$ ile topla → $\odot h$ → y

Elle örnekteki $\hat{W}$ ile, x = [1; 2; 0; −1]

#	işlem	sonuç
1	$g \odot x$ (sütun scale'leri)	[1,36; 1,61; 0; −0,80]
2	± topla: (+ − + +), (+ + + −)	[−1,05; 3,78]
3	$\ell \odot$, ℓ = [2,84; 0,53]	[−2,98; 2,01]
4	± topla: U_sign'ın satırları	[−0,97; −4,98; 4,98; −0,97]
5	$h \odot$, h = [0,57; 0,99; 0,60; 1,31]	y = [−0,55; −4,94; 2,97; −1,26]

Bir 4096 × 4096 katmanı, 0,1 BPW (r = 86, iki yol)

	FP16	LittleBit
ağır işlem	16,8 M MAC	1,41 M ±
çarpma	16,8 M	16 556
bellek	33,6 MB	209 KB

Hesaplar bizim, makalenin formülüyle.

$$y = h \odot (U_sign (\ell \odot (V_sign^T (g \odot x))))$$

Turuncu adımlarda hiç çarpma yok: sign(v) · a = ±a. Makale: A100'de 1-bit kernel ile 11,6 kata kadar (kernel düzeyi).

İkinci bir yol ilk yolun hatasını yakalar; kaç desen kullanılacağı, hedef BPW'den hesaplanır

Residual compensation

$$\hat{W} = \hat{W}_{\text{pri}} + \hat{W}_{\text{res}}, \quad W_{\text{res},0} = W - \hat{W}_{\text{pri},0}$$

İlk yaklaşımın hatasına, aynı türden ikinci bir yaklaşım eklenir; kendi işaretleri ve kendi scale takımı (h, g, ℓ) vardır. Kalan hataya Dual-SVID uygulanarak başlatılır; iki yol birlikte eğitilir.

Bit bütçesi ve rank

$$b = [2r(d_o + d_i) + 32(d_o + d_i) + 32r] \,/\, (d_o\, d_i)$$

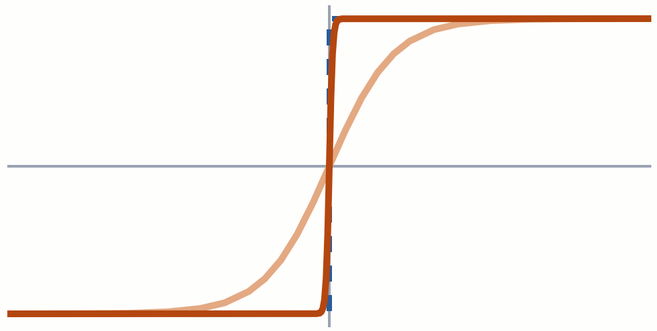
$$r = [b\, d_o\, d_i - 32(d_o + d_i)] \,/\, [2(d_o + d_i) + 32]$$

Baştaki 2: iki yol. İşaretler $r(d_o + d_i)$ bit, scale'ler $16(d_o + d_i + r)$ bit. Makalenin örneği: 4096 × 4096, $b = 0{,}55 \rightarrow r = 546$.

katman (aynı formül)	desen, yol	BPW
4 × 4 (elle örnek)	$r = 2$, tek yol	11
4096 × 4096	$r = 86$, iki yol	0,1
4096 × 11 008 (makale)	$r = 133$, iki yol	0,1

Okunuşu: küçük matriste scale'ler baskındır ve kazanç yoktur; matris büyüdükçe bit başına maliyet hızla düşer.

Training: sign’ın türevi sıfırdır; LittleBit geri yayılımda onu yumuşak bir rampa gibi davranarak öğrenir



sign(x) (kesikli), **tanh(100x)**, karşılaştırma için tanh(5x) (açık), $x \in [-1, 1]$.

Şekil: tanh(100x) neredeyse basamak ama sıfır civarında türevlenebilir.

SmoothSign

ileri: $\text{sign}(x)$ · geri: $k(1 - \tanh^2(kx))$, $k = 100$

sign’ın türevi her yerde 0’dır; gradient U ve V’ye ulaşamaz, işaretler hiç değişmez. Klasik çözüm STE: türevi 1 saymak. Yazarlar SmoothSign’ı ultra düşük bitte daha iyi buluyor.

Knowledge distillation

$\mathcal{L} = \mathcal{L}_{\text{out}} + \lambda\, \mathcal{L}_{\text{inter}}$, $\lambda = 10$

Teacher orijinal FP16 model, student LittleBit modeli. Student, teacher’ın çıkış dağılımını (KL divergence) ve ara katman çıktılarını (MSE) taklit eder.

Training boyunca U ve V gerçek sayı olarak tutulur, forward’da yalnızca işaretleri kullanılır. Veri: WikiText-2 ve C4 parçaları; 5 epoch, dizi uzunluğu 2048.

Sonuçlar: model 0,1 BPW'de bile çalışıyor, ama 0,3'ten 0,1'e geçişte kalite belirgin düşüyor

BPW	LittleBit 7B	LittleBit 13B	STBLLM 7B
16 (FP16)	5,47	4,88	—
1,0	9,08	8,18	—
0,8	9,53	8,52	13,81
0,7	9,85	8,76	19,17
0,55	10,47	9,24	30,67
0,3	12,00	10,48	1 800
0,1	**15,92**	**15,09**	—

Perplexity = e^loss: model her adımda ortalama kaç aday arasında kararsız. FP16'da ~5,5, 0,1 BPW'de ~16 aday. WikiText-2, Llama2; kaynak: makale, Tablo 1 (yeniden üretmedik).

26,06 → 0,84 GB

Llama2-13B, 0,1 BPW'de 31,0 kat küçük. 70B: 138,04 → 1,98 GB (69,7×).

11,6× kernel

A100'de 1-bit GEMV kernel'i, FP16'ya göre; uçtan uca inference değil.

Dikkat edilecekler

FP16'ya göre perplexity üç kat (5,47 → 15,92). 1 bitte BinaryMoS daha iyi (7,74). Residual yol, OPT-1.3B'de 0,1 BPW'de zarar veriyor (+11,5).

FPGA için: çarpıcısız matris-vektör çarpımı çok uygun, ama küçük katmanlarda scale'lerin bit maliyeti baskın

Fırsatlar

- ±1 ile çarpma toplama ya da çıkarmadır: ağır iş DSP yerine LUT'larla yapılır.
- İşaretler 1 bit: BRAM'e INT8'e göre 8 kat yoğun paketlenir.
- Hesap yine matris-vektör çarpımı: tek-token generation akışıyla uyumlu.
- ~630 KB BRAM, 0,1 BPW'de ~50 M weight'e yer açar (teorik).

Zorluk: scale'lerin payı

yalnız h ve g: $32(d_o + d_i) / (d_o\, d_i)$ BPW

katman	scale BPW
48 × 48 (gpt-nano)	**1,33**
512 × 512	0,125
4096 × 4096	0,016

gpt-nano'da scale'ler tek başına 0,1 BPW hedefini aşıyor.

Araştırma soruları

1 · Küçük modellerde LittleBit'in anlamlı BPW alt sınırı nedir? Makaledeki en küçük model OPT-1.3B.

2 · Binary matris-vektör birimi FPGA'de kaç LUT tüketir, token başına kaç cycle sürer?

3 · FP16 scale'ler integer'a indirilip bit-exact bir donanım kurulabilir mi?

ÖZET

1 · minGPT, GPT-2 mimarisinin sade hâli: embedding, attention ve MLP’den oluşan L tane block ve bir lm_head.

2 · Embedding bir one-hot çarpımı, yani tablo okuması; gradient yalnızca görülen token’ların satırlarına gider. Attention ise geçmişin içeriğe göre ağırlıklı ortalaması.

3 · Bu yapı FPGA’ye uygun: küçük, düzenli ve öngörülebilir. Zorluklar: non-linear işlemler, floating point ve büyüyen attention.

4 · LittleBit, weight’leri low-rank binary factor’lara ayırarak 0,1 BPW’ye iniyor; çarpıcısız matris-vektör çarpımı FPGA için doğal bir araştırma hedefi.

Teşekkürler. Sorular?

github.com/Muratatceken/mingpt-fpga